UNIVERSIDADE CIDADE DE SÃO PAULO – UNICID
CURSO DE CIÊNCIA DA COMPUTAÇÃO

ALEXANDRO DE ARAUJO JUNIOR
CAIQUE DOS SANTOS ALVES
DUQUESNIO DANIEL BANDESSA
VICTOR ALVES SOUZA DOS SANTOS

N-RAINHAS COM BACKTRACKING EM PROLOG:

solução no paradigma lógico e comparação com C e Java

SÃO PAULO
2026

ALEXANDRO DE ARAUJO JUNIOR – RGM 34625127
CAIQUE DOS SANTOS ALVES – RGM 39165566
DUQUESNIO DANIEL BANDESSA – RGM 33856885
VICTOR ALVES SOUZA DOS SANTOS – RGM 34696466

N-RAINHAS COM BACKTRACKING EM PROLOG:

solução no paradigma lógico e comparação com C e Java

Trabalho apresentado à disciplina Paradigmas de Linguagens de Programação, do curso de Ciência da Computação da Universidade Cidade de São Paulo (UNICID), como requisito parcial da avaliação A2 (tema L2, Grupo 04).

Professor: Cid Rodrigues de Andrade.

SÃO PAULO
2026

RESUMO

Este trabalho apresenta a solução do problema das N-rainhas no paradigma lógico, com a linguagem Prolog, executada no ambiente online SWISH. O problema consiste em posicionar N rainhas em um tabuleiro N×N sem que duas delas se ataquem e é um exemplo clássico de busca com retrocesso (*backtracking*). Foram desenvolvidas três versões que usam o mesmo mecanismo de busca nativo do Prolog: gerar e testar, *backtracking* com poda e verificação adiante com a heurística *first-fail*. A terceira encontra uma solução para 100 rainhas em cerca de 0,3 s e foi testada até N = 400. O programa também desenha o tabuleiro no navegador, mostra o *backtracking* passo a passo e executa 24 testes automáticos. A solução é comparada com implementações equivalentes em C e Java quanto à sintaxe, à semântica, ao gerenciamento de memória e aos *trade-offs*. Conclui-se que, em Prolog, a busca com retrocesso faz parte da própria linguagem: o programador declara o que é uma solução válida, e o motor de inferência explora as alternativas e desfaz as ligações sozinho. O custo é o desempenho, cerca de cem vezes menor que o de C e Java com o mesmo algoritmo, e a dependência da ordem dos objetivos para a eficiência.

Palavras-chave: Prolog. Programação lógica. Backtracking. N-rainhas. Paradigmas de linguagens de programação.

SUMÁRIO

1 INTRODUÇÃO 4
2 O PARADIGMA LÓGICO E O PROLOG 5
2.1 Fatos, regras e consultas 5
2.2 Unificação 5
2.3 Resolução e backtracking 5
3 DESCRIÇÃO DA SOLUÇÃO 6
3.1 Representação do tabuleiro 6
3.2 Versão 1: gerar e testar 6
3.3 Versão 2: backtracking com poda 7
3.4 Versão 3: verificação adiante e first-fail 8
3.5 Apresentação dos resultados 10
3.6 Testes automáticos 11
4 COMO EXECUTAR 13
5 ENTRADA E SAÍDA DE EXEMPLO 14
6 ANÁLISE COMPARATIVA 17
6.1 Sintaxe 17
6.2 Semântica 18
6.3 Gerenciamento de memória 19
6.4 Trade-offs 21
7 LOG DE USO DE INTELIGÊNCIA ARTIFICIAL 23
8 CONSIDERAÇÕES FINAIS 24
REFERÊNCIAS 25

1 INTRODUÇÃO

O problema das oito rainhas foi proposto em 1848 por Max Bezzel e depois generalizado para N rainhas: dispor N rainhas em um tabuleiro N×N de modo que nenhuma delas compartilhe linha, coluna ou diagonal com outra (BELL; STEVENS, 2009). Há solução para N = 1 e para todo N ≥ 4. O número de soluções, porém, cresce muito depressa: são 92 para N = 8, 14.200 para N = 12 e mais de 39 bilhões para N = 20 (OEIS FOUNDATION INC., 2026). Por isso o problema é usado há décadas como exemplo de busca com retrocesso. Wirth (1971), por exemplo, o escolheu para ilustrar o desenvolvimento de programas por refinamentos sucessivos.

No paradigma lógico, um programa é um conjunto de fatos e regras, e executá-lo é fazer uma consulta que o motor de inferência tenta provar (STERLING; SHAPIRO, 1994). Quando uma tentativa de prova falha, o Prolog volta sozinho ao último ponto em que havia uma alternativa e tenta a seguinte. É exatamente o comportamento de que as N-rainhas precisam: tentar uma posição, seguir enquanto for possível e voltar atrás quando não for. O enunciado do tema L2 destaca esse contraste: o *backtracking*, que em C ou Java precisa ser programado à mão, é nativo da linguagem.

O objetivo deste trabalho é resolver o problema das N-rainhas em Prolog de forma idiomática, sem imitar em Prolog as estruturas de controle de uma linguagem imperativa, e comparar a solução com versões equivalentes em C e Java. O código roda no SWISH (WIELEMAKER; LAGER; RIGUZZI, 2015), um ambiente online que dispensa instalação.

A seção 2 resume os conceitos do paradigma lógico usados na solução. A seção 3 descreve a solução e as três versões do algoritmo, e a seção 4 explica como executá-la. A seção 5 traz entradas e saídas copiadas de execuções reais. A seção 6 apresenta a análise comparativa com C e Java, a seção 7 registra o uso de inteligência artificial e a seção 8 traz as considerações finais.

2 O PARADIGMA LÓGICO E O PROLOG

2.1 Fatos, regras e consultas

Um programa Prolog é formado por cláusulas. Um fato afirma algo sem condições; uma regra, escrita `Cabeça :- Corpo`, afirma que a cabeça é verdadeira se o corpo for verdadeiro, e a vírgula do corpo significa “e”. Uma consulta pergunta se um objetivo pode ser provado e, se tiver variáveis, para quais valores (BRATKO, 2012). No programa deste trabalho, a regra `segura/1`, por exemplo, diz que uma lista de rainhas é segura se a primeira não ataca as demais e se o resto da lista também é seguro (Código 1).

2.2 Unificação

A unificação é a operação que torna dois termos iguais, ligando variáveis quando necessário. Ela ocupa o lugar da atribuição: uma variável lógica, uma vez ligada, não muda de valor naquele ramo da prova. É também a unificação que passa parâmetros e devolve resultados. Na consulta `rainhas(4, Qs)`, a variável `Qs` sai ligada à lista `[3,1,4,2]`. Expressões aritméticas não são calculadas pela unificação: `Q1 + D` é apenas um termo, e só predicados como `is/2`, `=:=/2` e `=\=/2` o avaliam.

2.3 Resolução e backtracking

Para provar uma consulta, o Prolog resolve os objetivos da esquerda para a direita e tenta as cláusulas na ordem em que aparecem no programa, em profundidade. Sempre que um objetivo pode ser provado de mais de uma forma, o sistema registra um ponto de escolha. Se mais adiante algum objetivo falha, a execução volta ao ponto de escolha mais recente, desfaz as ligações feitas depois dele e tenta a alternativa seguinte. Esse retrocesso cronológico é o *backtracking* nativo da linguagem (STERLING; SHAPIRO, 1994). Predicados da biblioteca como `member/2`, `select/3` e `permutation/2` deixam um ponto de escolha com uma alternativa para cada resposta possível. O botão Next do SWISH simplesmente pede ao sistema que retroceda e procure a próxima solução.

3 DESCRIÇÃO DA SOLUÇÃO

O código completo está no arquivo `src/nrainhas.pl` do repositório e é o mesmo carregado no *permalink* do SWISH. Ele tem nove partes: a regra de ataque, as três versões do algoritmo, consultas auxiliares, o desenho em texto, o desenho gráfico, os comandos para o usuário, a validação da entrada e os testes automáticos.

3.1 Representação do tabuleiro

Uma solução é uma lista `Qs = [Q1, Q2, ..., QN]` em que `Qi` é a linha da rainha que está na coluna `i`. Como há exatamente um elemento por coluna e a lista é uma permutação de 1 a N, duas rainhas nunca ficam na mesma coluna nem na mesma linha: essa parte das regras do problema é garantida pela própria representação. Resta conferir as diagonais. Duas rainhas estão na mesma diagonal quando a diferença entre as suas linhas é igual à diferença entre as suas colunas.

O predicado `nao_ataca/3` expressa essa condição (Código 1). A rainha `Q` não ataca as rainhas da lista se não estiver na mesma diagonal da primeira, que fica a `D` colunas de distância, nem na de nenhuma das seguintes. Não há laço: a repetição é feita por recursão sobre a lista, e a cláusula `nao_ataca(_, [], _)` diz que nenhuma rainha é atacada por uma lista vazia. Também não há valor de retorno: `nao_ataca/3` é verdadeiro ou falso e, quando é falso, provoca o *backtracking*.

Código 1 – Regras de ataque nas diagonais

```
nao_ataca(_, [], _).
nao_ataca(Q, [Q1|Qs], D) :-
    Q =\= Q1 + D,
    Q =\= Q1 - D,
    D1 is D + 1,
    nao_ataca(Q, Qs, D1).

segura([]).
segura([Q|Qs]) :-
    nao_ataca(Q, Qs, 1),
    segura(Qs).
```

Fonte: elaborado pelos autores (2026).

3.2 Versão 1: gerar e testar

A primeira versão traduz quase literalmente a definição do problema: uma solução é uma permutação das linhas 1 a N que seja segura (Código 2).

`permutation/2` gera um tabuleiro completo e `segura/1` o testa. Se o teste falha, o Prolog retrocede até `permutation/2`, que produz a próxima permutação. A versão é correta e curta, mas pode examinar até N! tabuleiros, ou seja, 3.628.800 para N = 10. Na prática, ela só é usável até N ≈ 10.

Código 2 – Versão 1: gerar e testar

```
rainhas_v1(N, Qs) :-
    numlist(1, N, Linhas),
    permutation(Linhas, Qs),
    segura(Qs).
```

Fonte: elaborado pelos autores (2026).

3.3 Versão 2: backtracking com poda

A segunda versão coloca uma rainha por coluna e testa cada uma assim que ela é colocada (Código 3). `select/3` escolhe uma linha ainda livre e cria um ponto de escolha; `nao_ataca/3` confere a nova rainha contra as que já estão no tabuleiro. Se a conferência falha, o Prolog volta ao `select/3` e tenta a próxima linha. Se nenhuma linha serve, a falha se propaga e a execução volta para a coluna anterior. Assim, um ramo inválido é abandonado logo no início, sem que o tabuleiro inteiro seja montado.

Código 3 – Versão 2: backtracking com poda

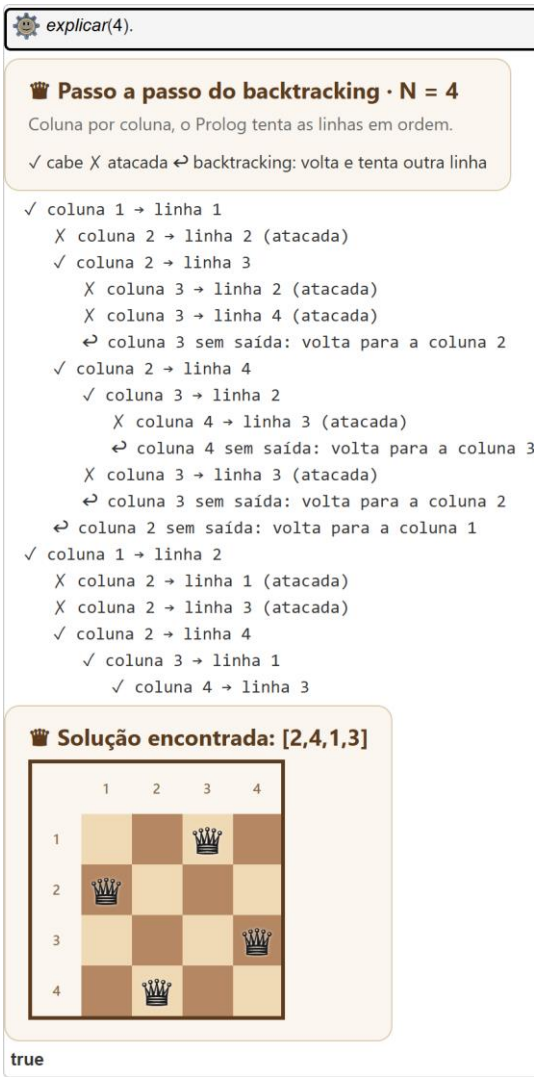
```
rainhas_v2(N, Qs) :-
    numlist(1, N, Linhas),
    posicionar(Linhas, [], Qs).

posicionar([], Colocadas, Qs) :-
    reverse(Colocadas, Qs).
posicionar(Livres, Colocadas, Qs) :-
    select(Q, Livres, Resto),          % ponto de escolha
    nao_ataca(Q, Colocadas, 1),        % poda: falhou? volta ao select
    posicionar(Resto, [Q|Colocadas], Qs).
```

Fonte: elaborado pelos autores (2026).

A lista `Colocadas` guarda a rainha mais recente na frente, porque é contra ela que `nao_ataca/3` mede a distância 1; no fim, `reverse/2` põe a lista na ordem das colunas. O comando `explicar(N)` usa essa mesma lógica e imprime cada tentativa, o que deixa o *backtracking* visível (Figura 1). Na linha “coluna 3 sem saída”, todas as linhas da coluna 3 falharam, e o Prolog retrocede para a coluna 2 sem nenhuma instrução explícita para desfazer a jogada.

Figura 1 – Backtracking passo a passo com explicar(4)



Fonte: elaborada pelos autores a partir de execução no SWISH (2026).

A versão 2 encontra a primeira solução para N = 20 em cerca de 1,6 s, mas o tempo cresce de forma exponencial: para N = 28, ela já leva 49,5 s.

3.4 Versão 3: verificação adiante e first-fail

Para tabuleiros grandes, a versão principal usa a verificação adiante (*forward checking*) com a heurística *first-fail*, técnicas clássicas dos problemas de satisfação de restrições (HARALICK; ELLIOTT, 1980; RUSSELL; NORVIG, 2021). A busca continua sendo o *backtracking* nativo do Prolog; o que muda é a quantidade de ramos podados. Cada coluna ainda vazia guarda o seu domínio, ou seja, as linhas em que uma rainha ainda poderia ficar. A cada passo, o programa:

a) escolhe o lugar com menos opções, que pode ser a coluna de menor domínio ou a linha que cabe no menor número de colunas. Se a busca tiver de falhar, é melhor que falhe cedo, com a árvore ainda pequena;

b) escolhe uma das opções com `member/2`, que é o ponto de escolha;

c) remove dos domínios das outras colunas as linhas atacadas pela nova rainha (Código 4). Se algum domínio fica vazio, ou se alguma linha livre deixa de caber em qualquer coluna, o passo falha e o Prolog volta ao `member/2`.

Código 4 – Passo principal da versão 3 e poda dos domínios

```
preencher([], _, Posicoes, Posicoes).
preencher(Colunas, Meio, Acumulado, Posicoes) :-
    escolher_casa(Colunas, Meio, C, L, Outras),  % ponto de escolha
    podar(Outras, C, L, Outras1),                % falhou? volta à escolha
    preencher(Outras1, Meio, [C-L|Acumulado], Posicoes).

podar([], _, _, []).
podar([C1-D1|Cs], C, L, [C1-D2|Cs2]) :-
    Distancia is abs(C1 - C),
    exclude(ataca(L, Distancia), D1, D2),
    D2 \== [],
    podar(Cs, C, L, Cs2).

ataca(L, _, L1) :- L1 =:= L.               % mesma linha
ataca(L, D, L1) :- L1 =:= L + D.           % diagonal descendo
ataca(L, D, L1) :- L1 =:= L - D.           % diagonal subindo
```

Fonte: elaborado pelos autores (2026).

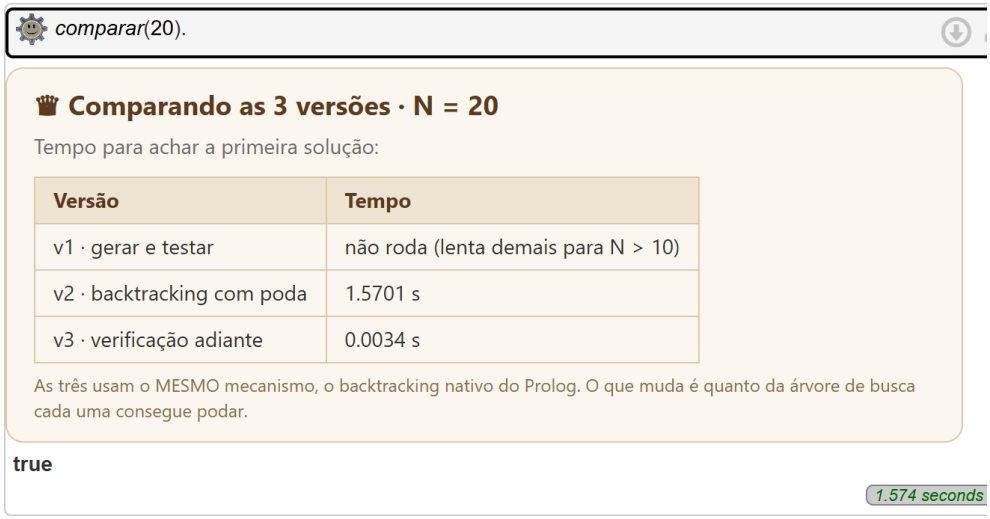
Colunas e linhas são tentadas do centro para as bordas. Em alguns valores de N, como 120, a primeira escolha leva a um ramo enorme sem saída. Para esses casos, a busca tem um teto de esforço, medido em inferências com `call_with_inference_limit/3`: se a primeira solução não aparece dentro do teto, a busca recomeça com outro critério de desempate (predicado `com_recomeco/4`). As soluções seguintes, pedidas com Next, continuam vindo da mesma busca, sem repetição.

O ponto mais importante para o paradigma é o que não precisou ser escrito. Quando o Prolog retrocede, os domínios voltam sozinhos ao estado anterior, porque são apenas listas ligadas a variáveis daquele ramo da prova. Em C ou Java, seria preciso guardar uma cópia dos domínios ou registrar cada remoção para desfazê-la depois.

Com essas mudanças, a primeira solução para N = 100 sai em cerca de 0,3 s. O comando `comparar(N)` mede as três versões lado a lado (Figura 2), e o Quadro 1 resume as diferenças. Nos testes feitos no SWISH em 27 de outubro de 2026, a

versão 3 encontrou uma solução válida para todo N de 1 a 250, com tempo médio de 1,3 s e pior caso de 5,6 s (N = 248), além de N = 300 (9,4 s) e N = 400 (24,5 s).

Figura 2 – Tempo das três versões para N = 20 com comparar(20)



Fonte: elaborada pelos autores a partir de execução no SWISH (2026).

Quadro 1 – Resumo das três versões

Versão	Predicado	Ponto de escolha	Ideia	Alcance
v1: gerar e testar	`rainhas_v1/2`	`permutation/2`	gera o tabuleiro inteiro e só depois testa	N ≈ 10
v2: backtracking com poda	`rainhas_v2/2`	`select/3`	testa cada rainha assim que ela é colocada	N ≈ 20 (N = 28 em 49,5 s)
v3: verificação adiante	`rainhas/2` (= `rainhas_v3/2`)	`member/2`	guarda as opções de cada coluna, começa pelo lugar mais restrito e recomeça se passar do teto	N = 100 em 0,3 s; testada até N = 400

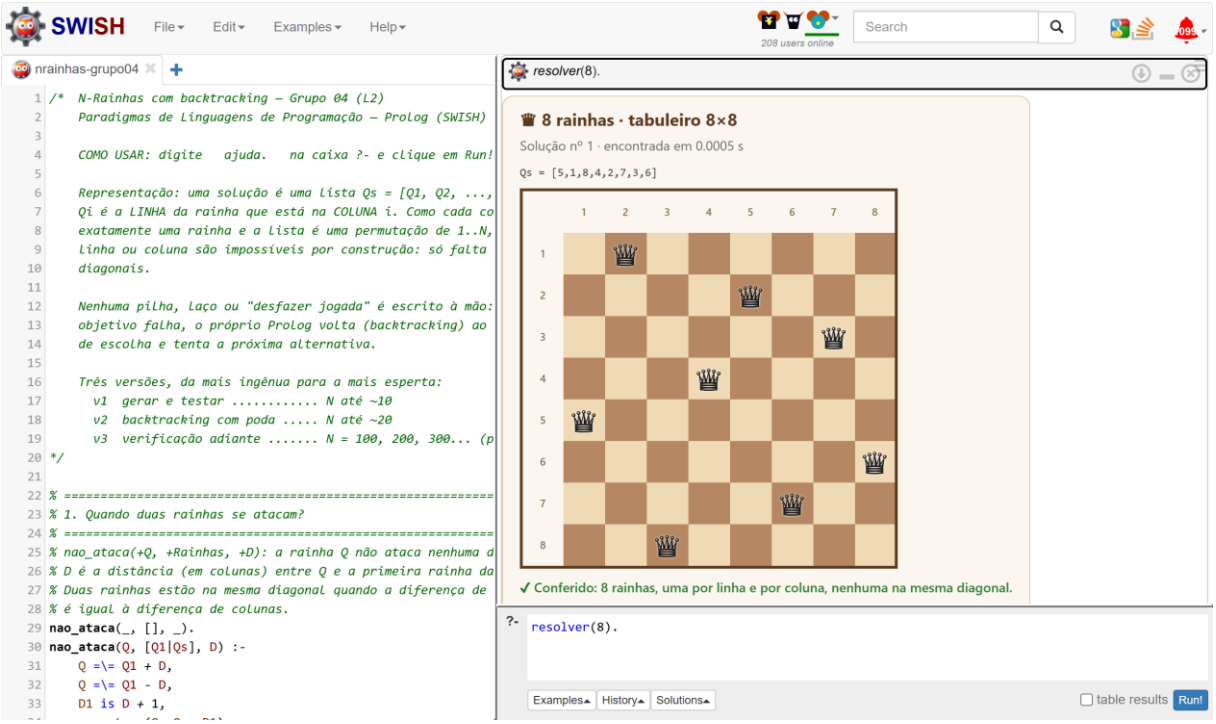
Fonte: elaborado pelos autores (2026).

3.5 Apresentação dos resultados

A lógica do problema não depende da forma de exibir as respostas. `tabuleiro/2` é uma relação pura entre uma solução e o seu desenho em texto. Para o desenho gráfico, o predicado `html/1` do SWISH recebe um termo Prolog que descreve a página, com elementos como `table`, `tr` e `td`. O tabuleiro, portanto, é um termo montado com `maplist/3`, como qualquer outro dado, e o tamanho de cada casa diminui conforme N cresce, para que o tabuleiro caiba na tela (Figura 3). A lista de comandos mostrada por `ajuda` e as regras de estilo do tabuleiro, que são textos CSS,

também ficam em bases de fatos (`comando/2` e `regra_css/1`). Cada solução exibida é conferida por `valida/1`, de forma independente do algoritmo que a gerou.

Figura 3 – Programa no SWISH e resposta da consulta resolver(8)



Fonte: elaborada pelos autores a partir de execução no SWISH (2026).

3.6 Testes automáticos

O comando `testar` executa 24 testes no próprio SWISH e mostra cada resultado assim que ele termina (Figura 4). Os testes formam uma base de fatos `teste(Grupo, Descricao, Caso, Esperado)`. O caso é um termo que diz o que medir, como `contagem(8)`, e o predicado `resultado/2` tem uma cláusula para cada tipo de caso. O teste passa quando o valor obtido é igual ao esperado. Por segurança, o SWISH não permite executar com `call/1` um objetivo montado durante a execução; por isso, os casos são dados, e não objetivos. Os seis grupos de testes conferem:

a) o próprio conferidor `valida/1`, com um tabuleiro correto e dois errados;

b) o total de soluções de N = 1 a 10, comparado com os valores conhecidos (OEIS FOUNDATION INC., 2026);

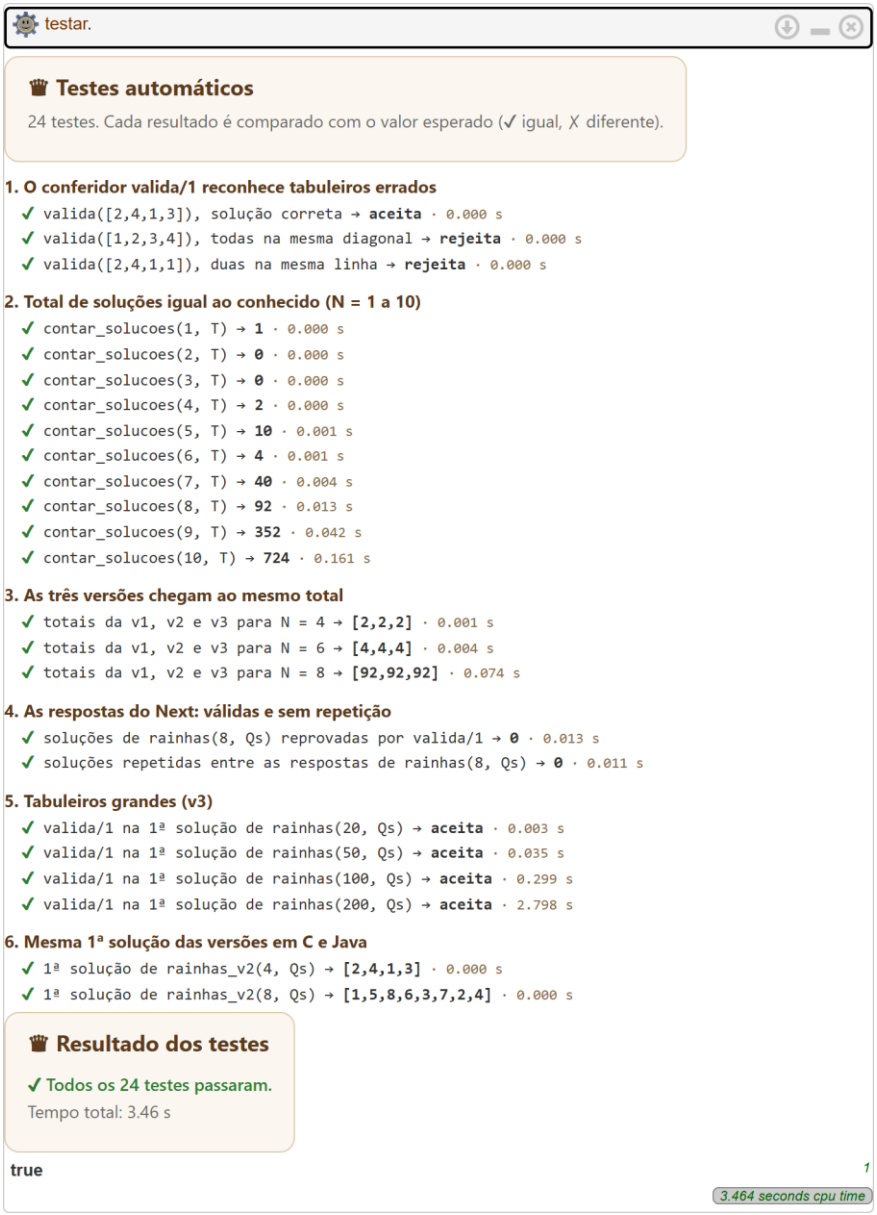
c) se as três versões chegam ao mesmo total para N = 4, 6 e 8;

d) se as 92 respostas para N = 8, obtidas por *backtracking*, são válidas e diferentes entre si;

e) as primeiras soluções para N = 20, 50, 100 e 200;

f) se a versão 2 dá a mesma primeira solução que os programas em C e Java.

Figura 4 – Execução dos testes automáticos com testar



Fonte: elaborada pelos autores a partir de execução no SWISH (2026).

4 COMO EXECUTAR

O programa roda no SWISH, o ambiente online homologado para Prolog, sem instalação nem cadastro. Para executá-lo:

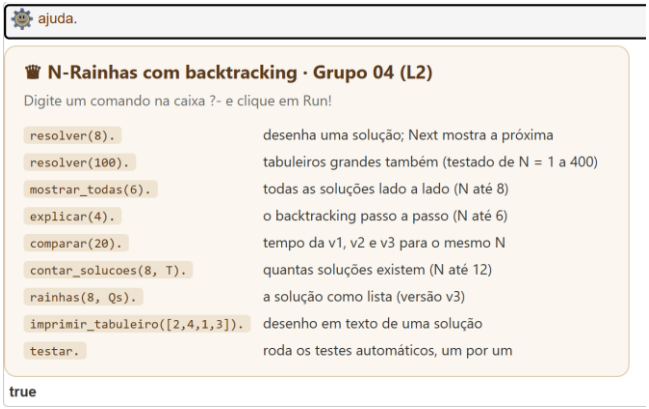
a) abra o *permalink* https://swish.swi-prolog.org/p/nrainhas-grupo04.pl;

b) o código aparece no painel da esquerda. No canto inferior direito, digite uma consulta na caixa `?-`, ou escolha uma em Examples, e clique em Run!;

c) comece pela consulta `ajuda.`, que lista os comandos disponíveis (Figura 5 e Quadro 2);

d) para ver outra solução, clique em Next: o Prolog retrocede e procura a próxima.

Figura 5 – Comandos disponíveis, mostrados pela consulta ajuda



Fonte: elaborada pelos autores a partir de execução no SWISH (2026).

Quadro 2 – Comandos do programa

Consulta	O que faz
`ajuda.`	mostra a lista de comandos
`resolver(N).`	resolve e desenha o tabuleiro N×N; Next mostra a próxima solução
`mostrar_todas(N).`	todas as soluções em miniatura, lado a lado (N até 8)
`explicar(N).`	mostra o *backtracking* passo a passo (N até 6)
`comparar(N).`	tempo das três versões para o mesmo N
`testar.`	roda os 24 testes automáticos
`contar_solucoes(N, Total).`	quantas soluções existem (N até 12)
`rainhas(N, Qs).`	a solução como lista (versão 3)
`rainhas_v1(N, Qs).` e `rainhas_v2(N, Qs).`	as versões mais simples, para comparação
`imprimir_tabuleiro(Qs).`	desenho da solução em texto

Fonte: elaborado pelos autores (2026).

5 ENTRADA E SAÍDA DE EXEMPLO

As saídas a seguir foram copiadas de execuções reais no SWISH, em 27 de outubro de 2026. Os tempos variam um pouco a cada execução. Em `Qs = [3, 1, 4, 2]`, cada posição da lista é uma coluna e cada valor é a linha da rainha: coluna 1 na linha 3, coluna 2 na linha 1, e assim por diante.

Código 5 – Consultas simples e respostas

```
?- rainhas(4, Qs).
Qs = [3, 1, 4, 2]
   Next: Qs = [2, 4, 1, 3]
   Next: false

?- rainhas_v2(4, [2,4,1,3]).
true

?- imprimir_tabuleiro([3,1,4,2]).
. Q . .
. . . Q
Q . . .
. . Q .
true

?- contar_solucoes(8, Total).
Total = 92

?- contar_solucoes(50, T).
⚠ contar_solucoes aceita N até 12: contar exige achar todas as soluções,
e N = 20 já tem mais de 39 bilhões. Para N grande, use resolver(N).
false
```

Fonte: elaborado pelos autores a partir de execução no SWISH (2026).

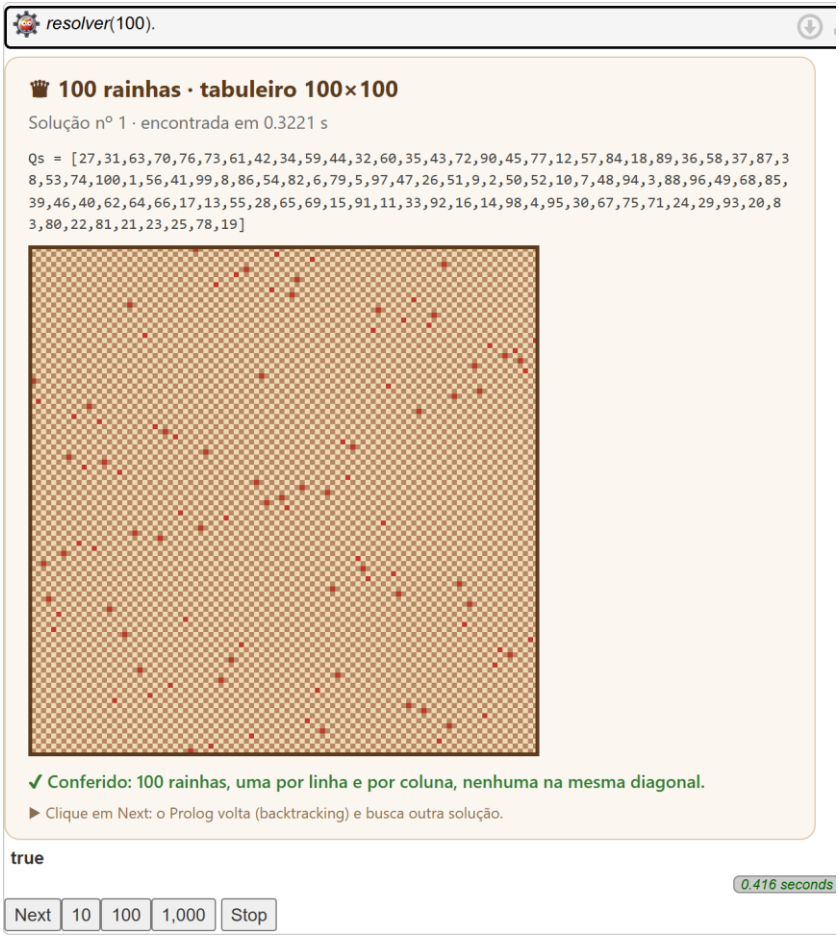
A consulta `rainhas_v2(4, [2,4,1,3])` mostra que o mesmo predicado que gera soluções também as confere, assunto retomado na seção 6.2. O Código 6 e a Figura 6 mostram a resposta para 100 rainhas, e o Código 7 traz a saída completa dos testes.

Código 6 – Saída de resolver(100)

```
?- resolver(100).
♛ 100 rainhas · tabuleiro 100×100
Solução nº 1 · encontrada em 0.3221 s
Qs =
[27,31,63,70,76,73,61,42,34,59,44,32,60,35,43,72,90,45,77,12,57,84,18,89,36,58,37,87,38,53,74,
100,1,56,41,99,8,86,54,82,6,79,5,97,47,26,51,9,2,50,52,10,7,48,94,3,88,96,49,68,85,39,46,40,62
,64,66,17,13,55,28,65,69,15,91,11,33,92,16,14,98,4,95,30,67,75,71,24,29,93,20,83,80,22,81,21,2
3,25,78,19]
[tabuleiro 100×100 desenhado: ver Figura 6]
✓ Conferido: 100 rainhas, uma por linha e por coluna, nenhuma na mesma diagonal.
▶ Clique em Next: o Prolog volta (backtracking) e busca outra solução.
true
```

Fonte: elaborado pelos autores a partir de execução no SWISH (2026).

Figura 6 – Tabuleiro 100×100 desenhado por resolver(100)



Fonte: elaborada pelos autores a partir de execução no SWISH (2026).

Código 7 – Saída completa de testar

```
?- testar.
♛ Testes automáticos
24 testes. Cada resultado é comparado com o valor esperado (✓ igual, ✗ diferente).
1. O conferidor valida/1 reconhece tabuleiros errados
  ✓ valida([2,4,1,3]), solução correta → aceita · 0.000 s
  ✓ valida([1,2,3,4]), todas na mesma diagonal → rejeita · 0.000 s
  ✓ valida([2,4,1,1]), duas na mesma linha → rejeita · 0.000 s
2. Total de soluções igual ao conhecido (N = 1 a 10)
  ✓ contar_solucoes(1, T) → 1 · 0.000 s
  ✓ contar_solucoes(2, T) → 0 · 0.000 s
  ✓ contar_solucoes(3, T) → 0 · 0.000 s
  ✓ contar_solucoes(4, T) → 2 · 0.000 s
  ✓ contar_solucoes(5, T) → 10 · 0.001 s
  ✓ contar_solucoes(6, T) → 4 · 0.001 s
  ✓ contar_solucoes(7, T) → 40 · 0.004 s
  ✓ contar_solucoes(8, T) → 92 · 0.013 s
  ✓ contar_solucoes(9, T) → 352 · 0.042 s
  ✓ contar_solucoes(10, T) → 724 · 0.161 s
3. As três versões chegam ao mesmo total
  ✓ totais da v1, v2 e v3 para N = 4 → [2,2,2] · 0.001 s
  ✓ totais da v1, v2 e v3 para N = 6 → [4,4,4] · 0.004 s
  ✓ totais da v1, v2 e v3 para N = 8 → [92,92,92] · 0.074 s
4. As respostas do Next: válidas e sem repetição
  ✓ soluções de rainhas(8, Qs) reprovadas por valida/1 → 0 · 0.013 s
  ✓ soluções repetidas entre as respostas de rainhas(8, Qs) → 0 · 0.011 s
5. Tabuleiros grandes (v3)
  ✓ valida/1 na 1ª solução de rainhas(20, Qs) → aceita · 0.003 s
  ✓ valida/1 na 1ª solução de rainhas(50, Qs) → aceita · 0.035 s
  ✓ valida/1 na 1ª solução de rainhas(100, Qs) → aceita · 0.299 s
  ✓ valida/1 na 1ª solução de rainhas(200, Qs) → aceita · 2.798 s
6. Mesma 1ª solução das versões em C e Java
  ✓ 1ª solução de rainhas_v2(4, Qs) → [2,4,1,3] · 0.000 s
  ✓ 1ª solução de rainhas_v2(8, Qs) → [1,5,8,6,3,7,2,4] · 0.000 s
♛ Resultado dos testes
  ✓ Todos os 24 testes passaram.
Tempo total: 3.46 s
true
```

Fonte: elaborado pelos autores a partir de execução no SWISH (2026).

6 ANÁLISE COMPARATIVA

Para a comparação, o algoritmo da versão 2 foi reescrito em C, do paradigma imperativo, e em Java, da orientação a objetos, nos arquivos `comparacao/nrainhas.c` e `comparacao/NRainhas.java`. As duas versões tentam as linhas na mesma ordem que o Prolog e chegam às mesmas respostas: o mesmo total de soluções (1, 0, 0, 2, 92 e 14.200 para N = 1, 2, 3, 4, 8 e 12) e a mesma primeira solução para N = 8, `[1,5,8,6,3,7,2,4]`. As diferenças discutidas a seguir vêm, portanto, do paradigma, e não do algoritmo. Python, também citado no enunciado, se comportaria como Java na maior parte dos pontos; a exceção é comentada na seção 6.2.

6.1 Sintaxe

Em Prolog, o programa é uma sequência de cláusulas no formato `Cabeça :- Corpo`. Não há declaração de tipos, chaves, atribuição nem laços: a vírgula significa "e", o ponto final encerra a cláusula e a repetição é feita por recursão. Variáveis começam com letra maiúscula, e átomos, com minúscula. Um mesmo predicado pode ter várias cláusulas, uma para cada caso, escolhidas pelo casamento de padrões na cabeça: `nao_ataca(_, [], _)` trata a lista vazia, e `nao_ataca(Q, [Q1|Qs], D)` separa a primeira rainha do resto da lista. Em C e Java, a mesma distinção é feita com `if` e com índices.

O Código 8 traz a parte central da versão em C, que corresponde ao Código 3 em Prolog. A versão em Prolog descreve o que é posicionar as rainhas: escolher uma linha livre que não seja atacada e posicionar as demais. A versão em C descreve como procurar: percorrer as linhas com um índice, marcar a casa, chamar a função recursivamente, desmarcar a casa e devolver um booleano. Sem contar comentários, o núcleo da versão 2 tem 15 linhas em Prolog (`nao_ataca/3`, `rainhas_v2/2` e `posicionar/3`), 45 em C (a estrutura `Tabuleiro`, as funções de alocação e liberação e as funções `livre`, `marcar` e `posicionar`) e 39 em Java (Código 9), que segue a estrutura do C organizada em métodos de uma classe.

Código 8 – Parte central da versão 2 em C

```
static bool livre(const Tabuleiro *t, int l, int c) {
    return !t->linha_usada[l]
        && !t->diag1[l - c + t->n - 1]
        && !t->diag2[l + c];
}

static void marcar(Tabuleiro *t, int l, int c, bool ocupada) {
    t->linha_usada[l] = ocupada;
    t->diag1[l - c + t->n - 1] = ocupada;
    t->diag2[l + c] = ocupada;
}

static bool posicionar(Tabuleiro *t, int c) {
    if (c == t->n)
        return true;
    for (int l = 0; l < t->n; l++) {
        if (livre(t, l, c)) {
            t->linha[c] = l;
            marcar(t, l, c, true);              /* tenta                */
            if (posicionar(t, c + 1))
                return true;
            marcar(t, l, c, false);             /* desfaz: backtracking */
        }
    }
    return false;                               /* nenhuma linha serve  */
}
```

Fonte: elaborado pelos autores (2026).

Código 9 – Método posicionar da versão em Java

```
private boolean posicionar(int c) {
    if (c == n) {
        return true;
    }
    for (int l = 0; l < n; l++) {
        if (livre(l, c)) {
            linhaDaColuna[c] = l;
            marcar(l, c, true);            // tenta
            if (posicionar(c + 1)) {
                return true;
            }
            marcar(l, c, false);           // desfaz: backtracking manual
        }
    }
    return false;                          // nenhuma linha serve
}
```

Fonte: elaborado pelos autores (2026).

6.2 Semântica

Unificação e atribuição. Em C e Java, `t->linha[c] = l` altera uma posição de memória, e o valor anterior se perde. Em Prolog, `=` é unificação: `Qs = [Q|Resto]` é verdadeiro se os dois lados puderem ficar iguais, e ligar uma variável não destrói nada, porque ela ainda não tinha valor. Uma variável ligada não muda mais naquele ramo da prova; o valor só “volta atrás” quando o Prolog retrocede e desfaz a ligação. Por

isso a versão 2 não precisa marcar e desmarcar casas: ao voltar, as listas `Livres` e `Colocadas` são exatamente as de antes da tentativa.

Forma de avaliação e controle. C e Java executam as instruções na ordem escrita, e o controle é explícito: `for`, `if` e `return`. Em Prolog também há uma ordem, com objetivos da esquerda para a direita e cláusulas de cima para baixo, mas o controle é implícito: é a falha de um objetivo que faz a busca voltar. A avaliação dos argumentos é estrita, como em C e Java, e não preguiçosa como em Haskell. As soluções, porém, são produzidas sob demanda: o Prolog só procura a segunda solução quando ela é pedida, pelo botão Next ou por `findall/3`. Em Python, o mais próximo disso seria uma função geradora com `yield`; em C e Java, foi preciso escrever uma segunda função, `contar` e `coletar`, para obter todas as soluções.

Parâmetros. Os predicados Prolog são relações, não funções, e seus argumentos não têm direção fixa. `rainhas_v2(4, Qs)` gera uma solução, e `rainhas_v2(4, [2,4,1,3])` confere se aquela lista é uma solução (Código 5). A função `posicionar` do C funciona em um único sentido: recebe a coluna, devolve `true` ou `false` e deixa o resultado em um vetor alterado como efeito colateral.

Tipos. Prolog tem tipagem dinâmica: não há declarações, e um erro de tipo, como comparar um átomo com um número em `=:=`, só aparece durante a execução. C tem tipagem estática, mas permite conversões implícitas, como tratar um inteiro como valor lógico; Java tem tipagem estática e forte, e os erros de tipo são detectados na compilação (SEBESTA, 2018).

Escopo. O escopo de uma variável Prolog é a cláusula em que ela aparece: o `Q` de `nao_ataca/3` não tem relação com o `Q` de `posicionar/3`. Não há variáveis globais, e tudo o que um predicado usa chega pelos argumentos. Em C, o estado da busca fica na estrutura `Tabuleiro`, passada por ponteiro e alterada por várias funções; em Java, fica em atributos privados do objeto, visíveis a todos os métodos da classe.

6.3 Gerenciamento de memória

Em C, a memória é gerenciada manualmente (Código 10): `criar` reserva os vetores com `malloc` e `calloc`, `destruir` os libera com `free`, e o programa precisa conferir se a alocação deu certo. Esquecer o `free` causa vazamento de memória, e usar um vetor depois de liberá-lo causa comportamento indefinido (KERNIGHAN; RITCHIE, 1988). Em compensação, o uso de memória é pequeno e previsível: quatro

vetores de tamanho proporcional a N e a pilha de chamadas da recursão, com profundidade N.

Código 10 – Alocação e liberação manual de memória em C

```
static bool criar(Tabuleiro *t, int n) {
    t->n = n;
    t->linha = malloc(n * sizeof *t->linha);
    t->linha_usada = calloc(n, sizeof *t->linha_usada);
    t->diag1 = calloc(2 * n - 1, sizeof *t->diag1);
    t->diag2 = calloc(2 * n - 1, sizeof *t->diag2);
    return t->linha && t->linha_usada && t->diag1 && t->diag2;
}

static void destruir(Tabuleiro *t) {
    free(t->linha);
    free(t->linha_usada);
    free(t->diag1);
    free(t->diag2);
}
```

Fonte: elaborado pelos autores (2026).

Em Java, os vetores são criados com `new` no *heap* e liberados pelo coletor de lixo da máquina virtual quando deixam de ser referenciados; não existe `free`. O programador ainda precisa cuidar do estado compartilhado: como o mesmo vetor é reaproveitado durante a busca, cada solução guardada por `todasSolucoes` é copiada com `clone()`.

Em Prolog, não há alocação explícita. As listas e os termos criados durante a prova ficam nas áreas de memória da máquina abstrata. O SWI-Prolog segue um modelo derivado da Máquina Abstrata de Warren (WAM): uma pilha global guarda os termos, uma pilha local guarda os ambientes das cláusulas e os pontos de escolha, e a trilha (*trail*) registra as ligações de variáveis que precisam ser desfeitas no retrocesso (AÏT-KACI, 1991; WIELEMAKER et al., 2012). Quando o Prolog volta a um ponto de escolha, desfaz as ligações registradas na trilha e descarta de uma vez tudo o que foi criado na pilha global depois daquele ponto. O retrocesso, portanto, desfaz a jogada e também recupera a memória, sem nenhum código do programador. A memória que sobrevive ao retrocesso é recolhida por um coletor de lixo.

Duas consequências aparecem no programa. Primeiro, a recursão de `nao_ataca/3` termina com a chamada recursiva, e a otimização de última chamada a executa sem fazer a pilha crescer, como se fosse um laço. Segundo, a versão 3 cria uma lista nova de domínios a cada rainha colocada, em vez de alterar vetores no lugar. Isso consome mais memória e explica parte do desempenho menor, mas é

também o que permite ao retrocesso restaurar os domínios antigos sem custo para o programador.

6.4 Trade-offs

Com o mesmo algoritmo, C e Java são cerca de cem vezes mais rápidos que o Prolog (Tabela 1). Eles usam vetores mutáveis e acesso direto à memória, enquanto o Prolog cria listas, unifica termos e registra pontos de escolha. Como os ambientes de medição foram diferentes, a comparação é aproximada. Ainda assim, ela mostra que, nas três linguagens, o *backtracking* simples cresce de forma exponencial: o C leva 9,3 s para N = 32 e não terminou N = 100 em 60 s. Quem resolve N = 100 em 0,3 s é o algoritmo da versão 3, e não a linguagem.

Tabela 1 – Tempo até a primeira solução

N	C (v2)	Java (v2)	Prolog v2	Prolog v3
20	0,017 s	0,028 s	1,6 s	0,003 s
24	0,035 s	0,055 s	5,4 s	–
28	0,30 s	0,34 s	49,5 s	0,003 s
32	9,3 s	–	–	–
100	mais de 60 s (interrompido)	–	–	0,3 s

Fonte: elaborada pelos autores (2026). C e Java medidos em computador pessoal (gcc 16.2 com -O2 e JDK 21); Prolog medido no SWISH. O traço indica valor não medido.

Para este problema, o paradigma lógico traz as seguintes vantagens:

a) o *backtracking* é nativo: não há código para desfazer jogadas, e uma fonte comum de erros, esquecer de desmarcar uma casa, simplesmente não existe;

b) o código é curto e próximo da definição do problema, o que facilita conferir que ele está correto;

c) todas as soluções vêm sem esforço extra: o botão Next, `findall/3` e `aggregate_all/3` reaproveitam a mesma busca;

d) os predicados são relações e servem tanto para gerar quanto para conferir soluções;

e) técnicas mais avançadas, como a verificação adiante da versão 3, ficam simples de escrever, porque o estado é restaurado no retrocesso.

As desvantagens são:

a) desempenho bem menor com o mesmo algoritmo e maior consumo de memória;

b) a eficiência depende da ordem dos objetivos e das cláusulas: as versões 1 e 2 descrevem as mesmas soluções, mas uma é muito mais lenta que a outra, e uma consulta como `contar_solucoes(50, T)` não terminaria (por isso o programa limita a contagem a N ≤ 12);

c) depurar é mais difícil, porque o fluxo de execução não está escrito no código: é preciso entender a ordem da busca e os pontos de escolha, e foi para mostrar isso que se criou o comando `explicar`;

d) os erros de tipo só aparecem durante a execução;

e) há pouco controle de baixo nível sobre a memória e a representação dos dados.

Para as N-rainhas, que são um problema de busca por natureza, o paradigma lógico se mostrou adequado: a parte difícil em C e Java, que é retroceder corretamente, é justamente a que o Prolog oferece pronta. Quando o desempenho é crítico e a busca é simples, C e Java continuam mais indicados. O Quadro 3 resume a comparação.

Quadro 3 – Resumo da comparação entre Prolog, C e Java

Aspecto	**Prolog**	**C**	**Java**
Tentativa e erro	ponto de escolha (`select/3`, `member/2`)	laço `for`	laço `for`
Voltar atrás	automático, quando um objetivo falha	manual, com `marcar(..., false)`	manual, com `marcar(..., false)`
Estado da busca	listas nos argumentos, sem alteração no lugar	vetores mutáveis em uma `struct`	vetores mutáveis em atributos privados
Todas as soluções	Next, `findall/3`, `aggregate_all/3`	segunda função (`contar`)	segundo método (`coletar`)
Tipos	dinâmicos	estáticos, com conversões implícitas	estáticos e fortes
Escopo das variáveis	a cláusula	o bloco; estado compartilhado pela `struct`	o bloco; estado nos atributos do objeto
Memória	pilhas da máquina Prolog, recuperadas no retrocesso, e coletor de lixo	`malloc`/`calloc` e `free` manuais	`new` e coletor de lixo da JVM
Linhas do núcleo da v2	15	45	39

Fonte: elaborado pelos autores (2026).

7 LOG DE USO DE INTELIGÊNCIA ARTIFICIAL

Conforme o item 2.4 do enunciado, o grupo usou inteligência artificial como apoio para aprender a linguagem e desenvolver o trabalho. A ferramenta foi o Claude Opus 5.5, da Anthropic, usado por meio do Claude Code. O Quadro 4 resume cada uso; o registro completo está no arquivo `log-ia.md` do repositório. Todo trecho gerado com apoio da IA foi testado no SWISH antes de ser aproveitado.

Quadro 4 – Registro de uso de inteligência artificial

Data	**O que foi pedido**	**O que foi aproveitado**	**O que foi reescrito ou entendido pelo grupo**
13/10/2026	Analisar o enunciado e o esqueleto do README e montar o plano do projeto	Plano de etapas (`PLANO.md`) e estrutura do README	a preencher pelo grupo
13/10/2026	Escrever o código Prolog em duas versões (gerar e testar; com poda) e testá-lo no SWISH	`nao_ataca/3`, `segura/1`, `rainhas_v1/2`, `rainhas_v2/2`, `tabuleiro/2`, `contar_solucoes/2`	a preencher pelo grupo
27/10/2026	Resolver N grande sem sair do estilo do Prolog, mostrar o tabuleiro de forma visual e escrever as versões em C e Java	Versão 3 (`rainhas_v3/2`), tabuleiro gráfico com `html/1`, comandos `ajuda`, `resolver`, `mostrar_todas`, `explicar` e `comparar`; `nrainhas.c` e `NRainhas.java`	a preencher pelo grupo
27/10/2026	Mostrar no SWISH os testes sendo executados	Comando `testar/0` (24 testes) e limite de N ≤ 12 em `contar_solucoes/2`	a preencher pelo grupo
27/10/2026	Redigir o rascunho deste relatório no formato ABNT e capturar as figuras do SWISH	Estrutura e texto-base do relatório; figuras da pasta `docs/prints`	a preencher pelo grupo

Fonte: elaborado pelos autores (2026).

8 CONSIDERAÇÕES FINAIS

O trabalho resolveu o problema das N-rainhas em Prolog usando o próprio mecanismo de busca da linguagem. As três versões mostram que, no paradigma lógico, melhorar o programa não significa escrever mais controle, e sim descrever melhor o problema. A versão 1 declara o que é uma solução; a versão 2 testa cada rainha assim que ela é colocada; a versão 3 acompanha as opções que restam e começa pelo lugar mais difícil. Em nenhuma delas o retrocesso foi programado. A versão principal resolve 100 rainhas em cerca de 0,3 s, foi testada até N = 400 e é conferida por 24 testes automáticos que rodam no próprio SWISH.

A comparação com C e Java confirmou o foco do tema L2. O *backtracking* que nessas linguagens exige laços, vetores de marcação e código para desfazer cada jogada é, em Prolog, parte da semântica da linguagem. O preço é o desempenho e a necessidade de entender a ordem da busca para escrever programas eficientes.

Como trabalhos futuros, o mesmo problema poderia ser resolvido com programação por restrições, pela biblioteca CLP(FD) do SWI-Prolog, ou com métodos de reparo local, como o *min-conflicts*, que encontram soluções para tabuleiros com milhões de rainhas (MINTON et al., 1992).

REFERÊNCIAS

AÏT-KACI, Hassan. **Warren's abstract machine**: a tutorial reconstruction. Cambridge, MA: MIT Press, 1991.

BELL, Jordan; STEVENS, Brett. A survey of known results and research areas for n-queens. **Discrete Mathematics**, [*s. l.*], v. 309, n. 1, p. 1-31, 2009.

BRATKO, Ivan. **Prolog programming for artificial intelligence**. 4. ed. Harlow: Pearson Education, 2012.

HARALICK, Robert M.; ELLIOTT, Gordon L. Increasing tree search efficiency for constraint satisfaction problems. **Artificial Intelligence**, [*s. l.*], v. 14, n. 3, p. 263-313, 1980.

KERNIGHAN, Brian W.; RITCHIE, Dennis M. **The C programming language**. 2. ed. Englewood Cliffs: Prentice Hall, 1988.

MINTON, Steven *et al*. Minimizing conflicts: a heuristic repair method for constraint satisfaction and scheduling problems. **Artificial Intelligence**, [*s. l.*], v. 58, n. 1-3, p. 161-205, 1992.

OEIS FOUNDATION INC. A000170: number of ways of placing n nonattacking queens on an n X n board. *In*: **THE ON-LINE Encyclopedia of Integer Sequences**. [*S. l.*], 2026. Disponível em: https://oeis.org/A000170. Acesso em: 27 out. 2026.

RUSSELL, Stuart; NORVIG, Peter. **Artificial intelligence**: a modern approach. 4. ed. Hoboken: Pearson, 2021.

SEBESTA, Robert W. **Conceitos de linguagens de programação**. 11. ed. Porto Alegre: Bookman, 2018.

STERLING, Leon; SHAPIRO, Ehud. **The art of Prolog**: advanced programming techniques. 2. ed. Cambridge, MA: MIT Press, 1994.

WIELEMAKER, Jan *et al*. SWI-Prolog. **Theory and Practice of Logic Programming**, Cambridge, v. 12, n. 1-2, p. 67-96, 2012.

WIELEMAKER, Jan; LAGER, Torbjörn; RIGUZZI, Fabrizio. SWISH: SWI-Prolog for sharing. *In*: INTERNATIONAL WORKSHOP ON USER-ORIENTED LOGIC PROGRAMMING, 1., 2015, Cork. **Proceedings** [...]. [*S. l.*: *s. n.*], 2015. Disponível em: https://arxiv.org/abs/1511.00915. Acesso em: 27 out. 2026.

WIRTH, Niklaus. Program development by stepwise refinement. **Communications of the ACM**, New York, v. 14, n. 4, p. 221-227, 1971.